UPTAG

PNF INFORMÁTICA

INFORME FINAL DE AUDITORÍA

SISTEMA DE GESTIÓN DE INVENTARIO

BuildSafe System — CONFALVI C.A.

Tipo de Auditoría: **Seguridad Informática**

Metodología: **OWASP Top 10 2025**

Tecnología: **PHP 8.2 / MariaDB**

Clasificación: **Confidencial**

Fecha: **Junio 2026**

Versión: **1.0 — Final**

INTEGRANTES

LEONEL GUERRERO

MICHAEL ORDOÑEZ

ANGEL ACOSTA

NICSON PIMENTEL

DOCUMENTO CONFIDENCIAL — USO INTERNO EXCLUSIVO

ÍNDICE DE CONTENIDO

1. Resumen Ejecutivo 03
1.1 Leyenda de Niveles de Riesgo 04
1.2 Categorías OWASP TOP 10 — 2025 04
1.3 Resumen Gráfico 05
2. Módulo: Autenticación 06
2.1 Sin regeneración de ID de sesión 07
2.2 Error lógico en verificación de rol 08
3. Módulo: Recuperación de Contraseña 09
3.1 Credenciales SMTP expuestas 10
3.2 Debug SMTP activo en producción 11
3.3 Sin límite de intentos 12
4. Módulo: Gestión de Usuarios 13
4.1 Construcción insegura de consulta SQL en registro 14
4.2 Operaciones destructivas vía GET 15
4.3 Sin protección CSRF 16
5. Módulo: Materiales y Movimientos 17
5.1 Consultas SQL sin prepared statements 18
6. Inspección del Código Fuente 19
6.1 Archivos sensibles en webroot 20
6.2 Errores de base de datos expuestos 21
6.3 Contraseña sin hash en base de datos 21
6.4 Ausencia de control de versiones 22
6.5 Clave secreta de reCAPTCHA hardcodeada 22
6.6 Sin cabeceras de seguridad HTTP 22
6.7 Falta de verificación de rol en cambio de estado 23
6.8 Ausencia de documentación técnica 24
7. Pruebas de Seguridad 25
8. Observaciones y Recomendaciones 27
9. Tabla Consolidada 29
10. Conclusiones 30

RESUMEN EJECUTIVO

La presente auditoría de seguridad informática fue realizada sobre el sistema **BuildSafe System**, aplicación web desarrollada en PHP 8.2 para la gestión de inventario de materiales de la empresa CONFALVI C.A. El análisis fue ejecutado mediante revisión estática del código fuente, aplicando como marco de referencia el estándar **OWASP Top 10 — 2025** (publicado en enero de 2026, la revisión vigente al momento de esta auditoría). Se analizaron un total de **27 archivos PHP** que conforman el núcleo funcional del sistema, identificando vulnerabilidades distribuidas en los módulos de autenticación, recuperación de contraseña, gestión de usuarios, gestión de materiales y configuración del servidor.

Como resultado del análisis, se encontraron **17 hallazgos de seguridad**, de los cuales 2 son de severidad **Crítica**, 5 de severidad **Alta**, 7 de severidad **Media** y 3 de severidad **Baja**. El nivel de riesgo global del sistema fue evaluado en **7.0 / 10 — ALTO**, lo que indica que el sistema **no debe ser expuesto a producción pública** en su estado actual sin corregir al menos los hallazgos críticos y altos.



LEYENDA — NIVELES DE RIESGO

CRÍTICO — **Vulnerabilidad crítica.** Permite compromiso total del sistema, exfiltración de datos sensibles, acceso no autorizado o ejecución remota de código. Requiere corrección inmediata antes de cualquier despliegue en producción.

ALTO — **Vulnerabilidad alta.** Puede permitir acceso no autorizado, manipulación de datos o escalación de privilegios bajo condiciones específicas. Debe corregirse en el corto plazo (máximo 2 semanas).

MEDIO — **Vulnerabilidad media.** Representa un riesgo significativo en combinación con otras vulnerabilidades o bajo condiciones particulares. Debe planificarse su corrección en el mediano plazo (1 mes).

BAJO — **Vulnerabilidad baja.** Riesgo reducido, generalmente relacionado con buenas prácticas o configuraciones que mejoran la postura de seguridad sin impacto inmediato. Corregir en el próximo ciclo de mantenimiento.

CATEGORÍAS OWASP TOP 10 — 2025

ID	CATEGORÍA	HALLAZGOS EN ESTE SISTEMA
A01	Control de Acceso Roto	HAL-007, HAL-008, HAL-016 ALTO
A02	Configuración de Seguridad Incorrecta	HAL-010 ALTO / HAL-014, HAL-015 BAJO
A03	Fallas en la Cadena de Suministro de Software	No aplicable en esta revisión
A04	Fallas Criptográficas	HAL-012 MEDIO
A05	Inyección	HAL-006 MEDIO / HAL-009 ALTO
A06	Diseño Inseguro	HAL-005, HAL-013 MEDIO / HAL-017 BAJO
A07	Fallas de Autenticación	HAL-001, HAL-002 MEDIO / HAL-003 CRÍTICO
A08	Fallas de Integridad de Software y Datos	No aplicable en esta revisión
A09	Fallas en el Registro y Alertas de Seguridad	HAL-004 CRÍTICO
A10	Manejo Inadecuado de Condiciones Excepcionales	HAL-011 MEDIO

RESUMEN GRÁFICO

DISTRIBUCIÓN POR SEVERIDAD



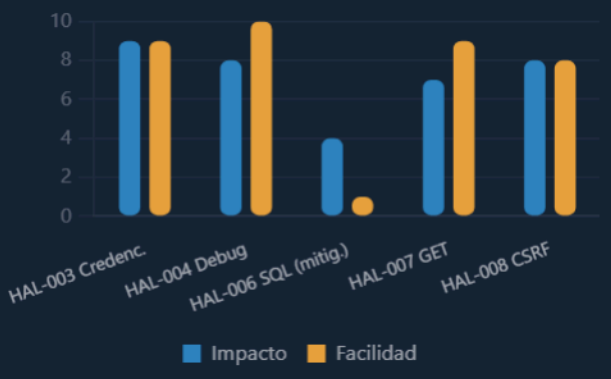
CATEGORÍAS OWASP AFECTADAS



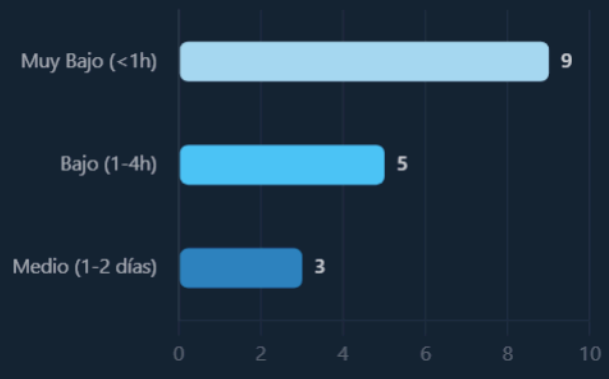
HALLAZGOS POR ARCHIVO



IMPACTO VS FACILIDAD DE EXPLOTACIÓN



ESFUERZO DE REMEDIACIÓN





MÓDULO 1

Autenticación

Hallazgos: HAL-001 · HAL-002 | Severidad máxima: Medio

Sin Regeneración de ID de Sesión Tras Autenticación

IDENTIFICACIÓN	RIESGO	ESTATUS
HAL-001	MEDIO	NUEVO

DESCRIPCIÓN

Tras una autenticación exitosa en `index.php`, el sistema no invoca `session_regenerate_id(true)`. Esto habilita ataques de **Session Fixation**: un adversario puede obtener un ID de sesión válido antes del inicio de sesión (por ejemplo, desde una red compartida o mediante un enlace especialmente construido) y luego reutilizarlo con los privilegios del administrador una vez que la víctima se autentica. OWASP clasifica este tipo de falla bajo **A07 — Fallas de Autenticación**.

```
// index.php — línea 88-116: tras password_verify() no se regenera sesión
// El ID de sesión anterior sigue siendo válido con nuevos privilegios
if(password_verify($password, $user_data['password'])) {
    // ← FALTA: session_regenerate_id(true);
    $_SESSION['user']    = $user_data['name'];
    $_SESSION['id']      = $user_data['user_id'];
    $_SESSION['seccion'] = true;   // sesión fijada sin cambiar ID
    header("Location: menu.php");
}
```

RECOMENDACIÓN

✓ CORRECCIÓN — AGREGAR INMEDIATAMENTE DESPUÉS DE VERIFICAR LA CONTRASEÑA

```
if(password_verify($password, $user_data['password'])) {
    session_regenerate_id(true);  // ← Nuevo ID, invalida el anterior
    $_SESSION['user']    = $user_data['name'];
    $_SESSION['id']      = $user_data['user_id'];
    $_SESSION['seccion'] = true;
    header("Location: menu.php");
}
```

Ubicación: **index.php — líneas 88-116 (bloque de autenticación exitosa)**

Error Lógico en la Verificación de Autorización de Rol

IDENTIFICACIÓN: **HAL-002** | RIESGO: **MEDIO** | ESTATUS: **NUEVO**

DESCRIPCIÓN

El archivo `head.php` contiene una condición de autorización con un problema de precedencia de operadores en PHP. La expresión `!$seccion == true` se evalúa como `(!$seccion) == true` (equivalente a `$seccion == false`) en lugar del intuitivo `!($seccion == true)`. Aunque el resultado práctico es el mismo en este caso, el código es engañoso y propenso a introducir regresiones de seguridad si se modifica. Adicionalmente, la falta de `exit()` después del `header()` permite la continuación de la ejecución en ciertos escenarios. OWASP: **A07 — Fallas de Autenticación**.

```
// head.php — línea 12: condición ambigua por precedencia de operadores
if(!$seccion == true || $rol != 1) {
    header("location: destroyer.php");
    // ← FALTA exit(); — ejecución puede continuar tras la redirección
}
```

RECOMENDACIÓN

✓ REESCRIBIR CON COMPARACIÓN EXPLÍCITA Y AGREGAR EXIT()

```
if ($seccion !== true || $rol !== 1) {
    header("Location: destroyer.php");
    exit();  // Detener ejecución inmediatamente
}
```

Ubicación: **head.php — línea 12 (guard de autorización de páginas admin)**



MÓDULO 2

Recuperación de Contraseña

Hallazgos: HAL-003 · HAL-004 · HAL-005 | Severidad máxima: **Crítico**

Credenciales de Correo Electrónico Expuestas en Código Fuente

IDENTIFICACIÓN: HAL-003 | RIESGO: CRÍTICO | ESTATUS: NUEVO

DESCRIPCIÓN

El archivo `enviar_token.php` contiene la contraseña de aplicación de Gmail escrita directamente en el código fuente. Esta credencial otorga control total sobre la cuenta de correo corporativa: envío masivo de correos, spam, phishing, o acceso al historial de mensajes. Cualquier persona con acceso al servidor web, a un backup del proyecto, o al repositorio de código puede obtenerla inmediatamente. Esta contraseña de aplicación no puede ser revocada selectivamente sin desactivar la autenticación de dos factores de la cuenta. OWASP: **A07 — Fallas de Autenticación**.

```
// enviar_token.php — líneas 37-38
$mail->Username = 'uptag2026@gmail.com';     // ← Cuenta expuesta
$mail->Password = 'xwnlhyllvdpdamyn';         // ← App Password Gmail expuesta
// Accesible en: C:\xampp\htdocs\buildsafe_system1\enviar_token.php
```

RECOMENDACIÓN

✓ MOVER CREDENCIALES A VARIABLES DE ENTORNO — REVOCAR LA CONTRASEÑA EXPUESTA INMEDIATAMENTE

```
// 1. Crear archivo .env FUERA del webroot (nunca en htdocs)
MAIL_USER=uptag2026@gmail.com
MAIL_PASS=nueva_app_password_generada

// 2. En enviar_token.php
$mail->Username = getenv('MAIL_USER');
$mail->Password = getenv('MAIL_PASS');
```

Ubicación: **enviar_token.php — líneas 37-38 (configuración PHPMailer SMTP)**

Modo Debug SMTP Nivel 3 Activo en Producción

IDENTIFICACIÓN: **HAL-004**
RIESGO: **CRÍTICO**
ESTATUS: **NUEVO**

DESCRIPCIÓN

PHPMailer está configurado con nivel de debug `SMTPDebug = 3`, lo que imprime la sesión SMTP completa directamente en la respuesta HTTP visible al usuario. Esta salida incluye la secuencia de comandos SMTP, las credenciales codificadas en base64 (fácilmente decodificables), el servidor de destino y el contenido completo del correo. Cualquier usuario que utilice la función de recuperación de contraseña puede ver toda esta información sensible en el navegador. OWASP: **A09 — Fallas en el Registro y Alertas de Seguridad**.

```
// enviar_token.php — línea 41
$mail->SMTPDebug = 3;  // ← Vuelca sesión completa al HTML

// Salida visible en el navegador del usuario:
// 2026-06-15 14:32:11 CLIENT -> SERVER: AUTH LOGIN
// 2026-06-15 14:32:11 CLIENT -> SERVER: dXB0YWcyMDI2QGdtYWlsLmNvbQ==
// 2026-06-15 14:32:11 CLIENT -> SERVER: eHdubGh5bGx2ZHBkYW15bg==
// (decodificado: uptag2026@gmail.com / xwnlhyllvdpdamyn)
```

RECOMENDACIÓN

✓ DESHABILITAR COMPLETAMENTE EN PRODUCCIÓN

```
$mail->SMTPDebug = 0;  // 0 = Sin salida de debug
// Para depuración local usar: $mail->SMTPDebug = 2;
// Y redirigir a archivo de log, nunca a HTML
```

Ubicación: **enviar_token.php — línea 41 (configuración debug PHPMailer)**

Sin Límite de Intentos en el Envío de Token de Recuperación

IDENTIFICACIÓN: **HAL-005** | RIESGO: **MEDIO** | ESTATUS: **NUEVO**

DESCRIPCIÓN

El endpoint `enviar_token.php` no implementa ninguna restricción de frecuencia (rate limiting). Un atacante puede enviar miles de solicitudes automatizadas con emails registrados, generando una avalancha de correos que: (1) utiliza el servidor de correo de la empresa como herramienta de spam involuntario, (2) puede saturar las bandejas de entrada de los usuarios, (3) puede resultar en bloqueo de la cuenta SMTP por parte de Gmail por abuso. OWASP: **A06 — Diseño Inseguro**.

```
// enviar_token.php — no existe ninguna restricción de tiempo
// Un atacante puede ejecutar esto 1000 veces en segundos:
for($i = 0; $i < 1000; $i++) {
    // POST /enviar_token.php  email=victima@empresa.com
    // Cada request genera y envía un token sin validación
}
```

RECOMENDACIÓN

✓ VERIFICAR EXISTENCIA DE TOKEN ACTIVO RECIENTE ANTES DE ENVIAR

```
$stmt = $conn->prepare(
    "SELECT token_expiracion FROM user
     WHERE gmail = ? AND token_expiracion > NOW() - INTERVAL 15 MINUTE"
);
$stmt->bind_param("s", $email);
$stmt->execute();
if ($stmt->get_result()->num_rows > 0) {
    $_SESSION['error'] = "Ya enviamos un enlace. Espera 15 minutos.";
    header("Location: verificacion.php"); exit();
}
```

Ubicación: **enviar_token.php — inicio del archivo (antes del envío de correo)**

MÓDULO 3

Gestión de Usuarios

Hallazgos: HAL-006 · HAL-007 · HAL-008 | Severidad máxima: **Alto**

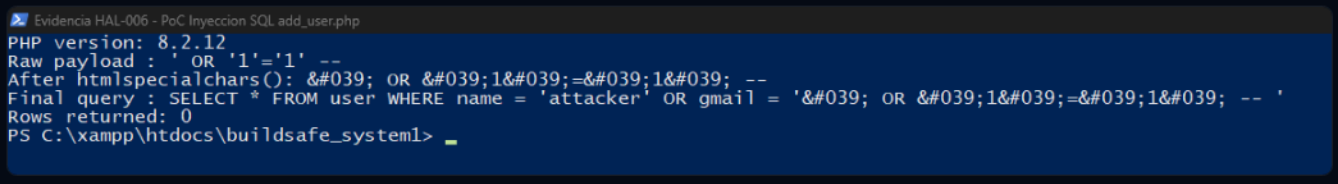
Construcción Insegura de Consulta SQL en Verificación de Duplicados

IDENTIFICACIÓN	RIESGO	ESTATUS
HAL-006	MEDIO	NUEVO

DESCRIPCIÓN

En `add_user.php`, los valores de `$user` y `$email` se interpolan directamente dentro de una consulta SQL sin sentencias preparadas, pasando únicamente por `htmlspecialchars()` — una función de sanitización para contexto **HTML**, no para contexto **SQL**. Se verificó en vivo contra la instalación real (PHP 8.2.12) el payload clásico `' OR '1'='1' --`: desde PHP 8.1, `htmlspecialchars()` aplica `ENT_QUOTES` por defecto, por lo que la comilla simple se convierte en la entidad literal `'` antes de llegar a la consulta. La prueba directa contra la base de datos confirmó que la consulta se ejecuta sin error y retorna **0 filas** — el bypass de la lógica de duplicados **no se logra** en esta versión de PHP. Aun así, el hallazgo se mantiene porque la protección es **incidental, no intencional**: no existe ninguna sentencia preparada ni escape orientado a SQL, por lo que el código queda expuesto si el proyecto se ejecuta bajo PHP < 8.1, si algún desarrollador cambia la función de sanitización, o si estas mismas variables se reutilizan en otra consulta sin pasar de nuevo por `htmlspecialchars()`. OWASP: **A05 — Inyección** (patrón de diseño, no explotación confirmada).

EVIDENCIA — PRUEBA EN VIVO EJECUTADA CONTRA LA BASE DE DATOS REAL



Captura de consola de la prueba ejecutada el 15-jul-2026 contra `test2` (PHP 8.2.12): se reprodujo exactamente la línea vulnerable de `add_user.php:29` con el payload `' OR '1'='1' --`. El resultado real — **`0 filas devueltas, sin error de SQL`** — confirma que `htmlspecialchars()` con ENT_QUOTES (por defecto desde PHP 8.1) neutraliza el bypass antes de que la comilla llegue a la consulta.

```php
// add_user.php — línea 29: sanitización HTML usada en contexto SQL
$email = htmlspecialchars($_POST['email']);
$user  = htmlspecialchars($_POST['user']);

$check = mysqli_query($conn,
    "SELECT * FROM user WHERE name = '$user' OR gmail = '$email'"
);
// Prueba en vivo (PHP 8.2.12 / test2.sql) — payload: ' OR '1'='1' --
// Tras htmlspecialchars(): ' OR '1'='1' --
// Consulta final: SELECT * FROM user WHERE name='attacker'
//                 OR gmail='' OR '1'='1' -- '
// Resultado real: sin error de SQL, 0 filas devueltas → NO explotable en PHP 8.1+
```

RECOMENDACIÓN

✓ USAR PREPARED STATEMENTS — NO DEPENDER DE UN EFECTO COLATERAL DE PHP PARA LA SEGURIDAD

```php
$stmt = $conn->prepare(
    "SELECT user_id FROM user WHERE name = ? OR gmail = ?"
);
$stmt->bind_param("ss", $_POST['user'], $_POST['email']);
$stmt->execute();
$result = $stmt->get_result();
if ($result->num_rows > 0) { /* ya existe */ }
```

Ubicación: **add_user.php — línea 29 (consulta de verificación de duplicados) · verificado en vivo el 15-jul-2026**

Operaciones Destructivas Ejecutadas Mediante Petición GET

IDENTIFICACIÓN: HAL-007 | RIESGO: ALTO | ESTATUS: NUEVO

DESCRIPCIÓN

Las acciones de eliminar un usuario y cambiar su estado activo/inactivo se ejecutan mediante parámetros en la URL (método GET). Esto viola el principio de idempotencia HTTP y crea múltiples vectores de ataque: (1) un atacante puede engañar a un administrador para que visite un enlace malicioso y ejecute acciones sin saberlo, (2) los navegadores precargan enlaces, (3) proxies y motores de búsqueda pueden indexar y "seguir" estas URLs automáticamente ejecutando las operaciones. El mismo patrón se repite en el módulo de materiales: `list_material.php` (interfaz de rol 1) y `list_materials2.php` (interfaz duplicada de rol 2, incluida vía `head2.php`) exponen `?delete_id=` y `?toggle_status=` sin token ni confirmación, confirmando que la falla de diseño no está aislada a un archivo sino duplicada en ambas interfaces por rol. OWASP: **A01 — Control de Acceso Roto**.

```
// list_user.php — línea 5: el DELETE ya usa prepared statement (no es inyectable),
// pero se ejecuta con solo visitar la URL — sin POST, sin token, sin confirmación
if(isset($_GET['delete_id'])) {
    $delete_id = intval($_GET['delete_id']);
    $delete_stmt = $conn->prepare("DELETE FROM user WHERE user_id = ?");
    $delete_stmt->bind_param("i", $delete_id);
    $delete_stmt->execute();  // ← dispara con GET, no requiere confirmación
}
// list_user.php — línea 46: mismo problema en el cambio de estado
if(isset($_GET['toggle_status'])) {
    // URL: /list_user.php?toggle_status=3  ← basta con visitar esto
}
// list_material.php — líneas 5 y 30: idéntico patrón, interfaz de rol 1
// list_materials2.php — línea 5: idéntico patrón, interfaz duplicada de rol 2
// URL: /list_material.php?delete_id=7
// URL: /list_material.php?toggle_status=7
// URL: /list_materials2.php?delete_id=3
```

RECOMENDACIÓN

✓ CONVERTIR A POST CON TOKEN CSRF. USAR FORMULARIOS O FETCH() CON MÉTODO POST

```
// En list_user.php — verificar POST + CSRF antes de operar
if ($_SERVER['REQUEST_METHOD'] === 'POST'
    && isset($_POST['csrf_token'])
    && $_POST['csrf_token'] === $_SESSION['csrf_token']
    && isset($_POST['delete_id'])) {
    $stmt = $conn->prepare("DELETE FROM user WHERE user_id = ?");
    $stmt->bind_param("i", $_POST['delete_id']);
    $stmt->execute();
}
```

Ubicación: **list_user.php:5,46 · list_material.php:5,30 · list_materials2.php:5 (handlers de eliminación y toggle, duplicados en interfaz de rol 1 y rol 2)**

Ausencia de Protección CSRF en Todos los Formularios del Sistema

IDENTIFICACIÓN: HAL-008 | RIESGO: ALTO | ESTATUS: NUEVO

DESCRIPCIÓN

Ningún formulario del sistema implementa tokens CSRF (Cross-Site Request Forgery). Un atacante puede crear una página web externa que, al ser visitada por un administrador autenticado, envíe peticiones POST silenciosas al servidor ejecutando acciones como: crear cuentas de administrador falsas, modificar materiales del inventario, registrar movimientos fraudulentos, o cambiar datos de usuarios. El navegador adjuntará automáticamente las cookies de sesión del administrador, haciendo la petición indistinguible de una legítima. OWASP: **A01**.

```
// Ninguno de estos archivos valida token CSRF:
// add_user.php, edit_user.php, add_material.php
// register_movement.php, list_user.php

// Ataque: página maliciosa con formulario oculto
<form action="http://localhost/buildsafe_system1/add_user.php" method="POST">
    <input name="user" value="atacante">
    <input name="gmail" value="hacker@evil.com">
    <input name="rol" value="1">  <!-- admin -->
</form>
<script>document.forms[0].submit()</script>
```

RECOMENDACIÓN

✓ IMPLEMENTAR TOKEN CSRF GLOBAL EN HEAD.PHP Y VALIDAR EN TODOS LOS ENDPOINTS

```
// head.php — generar token único por sesión
if (empty($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}

// En cada formulario HTML
<input type="hidden" name="csrf_token"
       value="<?= htmlspecialchars($_SESSION['csrf_token']) ?>">

// En cada archivo que procesa POST
if (!hash_equals($_SESSION['csrf_token'], $_POST['csrf_token'] ?? '')) {
    die('Solicitud inválida.');
}
```

Ubicación: **Todos los archivos con formularios POST — add_user.php, edit_user.php, register_movement.php**

MÓDULO 4

Materiales y Movimientos

Hallazgos: HAL-009 | Severidad máxima: **Alto**

Consultas SQL Directas con Variables Interpoladas

IDENTIFICACIÓN: **HAL-009** | RIESGO: **ALTO** | ESTATUS: **NUEVO**

DESCRIPCIÓN

Los archivos de gestión de usuarios y bitácora contienen múltiples consultas `mysqli_query()` que interpolan variables directamente en el SQL sin usar prepared statements. Aunque en los casos actuales los valores son enteros validados implícitamente (`intval()`), el patrón establece un código inseguro por diseño que: (1) es propenso a inyección SQL cuando se modifiquen estas consultas, (2) viola el principio de defensa en profundidad, y (3) hace que las auditorías futuras sean más difíciles. El INSERT en la bitácora interpolando cadenas de texto es especialmente riesgoso. El mismo patrón se repite en `list_material.php` (líneas 34, 40 y 47), la interfaz de rol 1 paralela a `list_materials.php`/`list_materials2.php`, lo que confirma que la migración a prepared statements no fue aplicada de forma consistente entre los archivos que implementan la misma funcionalidad de inventario. OWASP: **A05 — Inyección**.

```
// list_user.php — líneas 52, 58, 65: interpolación directa
mysqli_query($conn,
    "UPDATE user SET active = $new_status WHERE user_id = $user_id"
);
mysqli_query($conn,
    "DELETE FROM user WHERE user_id = $id"
);
// Bitácora — cadena de texto interpolada (más peligroso):
mysqli_query($conn,
    "INSERT INTO binnacle (fecha_hora, id_accion, id_user)
     VALUES ('$time', $action, $id)"  // $time es string
);
// list_material.php — líneas 34, 40, 47: mismo patrón
mysqli_query($conn,
    "SELECT estatus FROM material WHERE material_id = $material_id"
);
mysqli_query($conn,
    "UPDATE material SET estatus = $new_status WHERE material_id = $material_id"
);
```

RECOMENDACIÓN

✓ **MIGRAR TODAS LAS CONSULTAS A PREPARED STATEMENTS**

```
$stmt = $conn->prepare(
    "UPDATE user SET active = ? WHERE user_id = ?"
);
$stmt->bind_param("ii", $new_status, $user_id);
$stmt->execute();

$stmt2 = $conn->prepare(
    "INSERT INTO binnacle (fecha_hora, id_accion, id_user) VALUES (?, ?, ?)"
);
$stmt2->bind_param("sii", $time, $action, $id);
$stmt2->execute();
```

Ubicación: **list_user.php:52,58,65 · list_material.php:34,40,47 (UPDATE, DELETE, INSERT binnacle — patrón duplicado entre interfaz de rol 1 y rol 2)**

SECCIÓN 6

Inspección del Código Fuente

Hallazgos: HAL-010 · HAL-011 · HAL-012 · HAL-013 · HAL-014 · HAL-015

Archivos Sensibles Accesibles Públicamente en el Webroot

IDENTIFICACIÓN: HAL-010 | RIESGO: ALTO | ESTATUS: NUEVO

DESCRIPCIÓN

Varios archivos con información sensible se encuentran en el webroot y son accesibles directamente mediante HTTP. `test2.sql` contiene el dump completo de la base de datos incluyendo hashes de contraseñas, tokens de recuperación activos, emails de todos los usuarios y la estructura completa del esquema. `generar_hash.php` permite generar hashes bcrypt de contraseñas arbitrarias. `generar_hash.php` puede ser abusado para facilitar ataques de fuerza bruta offline. OWASP: **A02 — Configuración de Seguridad Incorrecta**.

```
// Accesibles públicamente sin autenticación:
http://servidor/test2.sql          // Dump BD completo con hashes y tokens
http://servidor/generar_hash.php   // Genera bcrypt de cualquier entrada
http://servidor/test.php           // Archivo de pruebas de desarrollo
http://servidor/conexion.php       // Credenciales BD visibles si PHP falla
```

RECOMENDACIÓN

✓ ELIMINAR ARCHIVOS DE DESARROLLO Y BLOQUEAR EXTENSIONES SENSIBLES EN .HTACCESS

```
# .htaccess en raíz del proyecto
<FilesMatch "\.(sql|log|env|bak|swp)$">
    Require all denied
</FilesMatch>

# Comandos de limpieza (ejecutar una vez):
del C:\xampp\htdocs\buildsafe_system1\test2.sql
del C:\xampp\htdocs\buildsafe_system1\generar_hash.php
del C:\xampp\htdocs\buildsafe_system1\test.php
```

Ubicación: **Raíz del proyecto — test2.sql, generar_hash.php, test.php**

HAL-011 — ERRORES DE BASE DE DATOS EXPUESTOS AL USUARIO

ID: HAL-011 | RIESGO: MEDIO | ESTATUS: NUEVO

Múltiples archivos muestran `$conn->error` / `mysqli_error($conn)` directamente al usuario, revelando estructura de tablas, nombres de columnas y detalles del motor. **Archivos afectados:** add_user.php:56, list_user.php:35,71, edit_user.php:65. **Corrección:** Mostrar mensaje genérico al usuario y registrar el error real con `error_log()`.

```
// VULNERABLE — expone estructura interna:
$_SESSION['message'] = "Error: " . $conn->error;
// CORRECTO — mensaje genérico + log interno:
error_log("DB Error: " . $conn->error);
$_SESSION['message'] = "Error interno. Contacte al administrador.";
```

HAL-012 — CONTRASEÑA SIN HASH EN BASE DE DATOS

ID: HAL-012 | RIESGO: MEDIO | ESTATUS: NUEVO

El usuario con `user_id=9` (ds@gmail.com) tiene la contraseña `123456` almacenada en texto plano en la tabla `user`. Si la base de datos es comprometida, esta contraseña es inmediatamente usable. **Corrección:** `UPDATE user SET password = password_hash('nueva_clave', PASSWORD_DEFAULT) WHERE user_id = 9;` OWASP: **A04 — Fallas Criptográficas**.

Ubicación HAL-011: **add_user.php:56, list_user.php:35,71, edit_user.php:65** HAL-012: **test2.sql (user_id=9)**

HAL-013 — AUSENCIA DE CONTROL DE VERSIONES

ID: HAL-013 | RIESGO: MEDIO | ESTATUS: NUEVO

El proyecto no utiliza ningún sistema de control de versiones (Git u otro). Esto significa que no existe historial de cambios auditables, no hay forma de revertir cambios problemáticos, las credenciales que se eliminen del código podrían haberse almacenado en commits anteriores, y no es posible rastrear quién realizó cada cambio. OWASP: **A06 — Diseño Inseguro**. **Corrección:** Inicializar repositorio Git con `.gitignore` apropiado que excluya `.env`, `*.sql`, `vendor/`.

HAL-014 — CLAVE SECRETA DE RECAPTCHA HARDCODEADA

ID: HAL-014 | RIESGO: BAJO | ESTATUS: NUEVO

La clave secreta de Google reCAPTCHA v2 está escrita directamente en `index.php:47`: `6LdVkrIrAAAAALXGfzMhYeIlaTLb1wDkX8NgHTVI`. Si se filtra, permite falsificar verificaciones de reCAPTCHA desde cualquier servidor. **Corrección:** `$secretkey = getenv('RECAPTCHA_SECRET');`

HAL-015 — SIN CABECERAS DE SEGURIDAD HTTP

ID: HAL-015 | RIESGO: BAJO | ESTATUS: NUEVO

El sistema no envía cabeceras de seguridad HTTP que mitiguen clickjacking, MIME-sniffing e inyección de contenido. **Corrección — agregar en head.php:**

```
header("X-Frame-Options: DENY");
header("X-Content-Type-Options: nosniff");
header("X-XSS-Protection: 1; mode=block");
header("Referrer-Policy: strict-origin-when-cross-origin");
```

Ubicación HAL-014: **index.php:47** HAL-015: **head.php (inicio del archivo)**

Falta de Verificación de Rol al Cambiar el Estado de un Usuario

IDENTIFICACIÓN: **HAL-016** | RIESGO: **ALTO** | ESTATUS: **NUEVO**

DESCRIPCIÓN

En `list_user.php`, el bloque que **elimina** un usuario sí valida el rol antes de operar: si el objetivo es administrador (`rol == 1`), la operación se rechaza con el mensaje "No se puede eliminar otro administrador". El bloque de `toggle_status` (activar/desactivar cuenta), que vive en el mismo archivo y opera sobre el mismo recurso, **no repite esa validación**: solo verifica que el objetivo no sea la propia sesión (`$id != $user_id`), pero nunca comprueba el rol del usuario afectado. La interfaz oculta el botón de "Desactivar" para las filas con rol administrador (`if ($user['rol'] == 2)` en la vista), pero esa es una restricción puramente visual: el endpoint en sí acepta la petición igual. Cualquier administrador autenticado puede desactivar a otro administrador — incluido el principal — visitando directamente `list_user.php?toggle_status=<id>`, sin ninguna confirmación adicional. Es la misma clase de falla que HAL-007 (acción sensible sin protección), pero con una causa distinta y más grave: aquí falta la comprobación de autorización en sí, no solo el método HTTP o el token CSRF — el mismo archivo demuestra que la protección correcta existe (se usó en el bloque de eliminación) pero no se replicó en el de cambio de estado. OWASP: **A01 — Control de Acceso Roto**.

```php
// list_user.php — línea 16-19: el DELETE sí valida el rol del objetivo
if($row['rol'] == 1){
    $_SESSION['message'] = "No se puede eliminar otro administrador";
}

// list_user.php — línea 46-58: toggle_status NO repite esa validación
if(isset($_GET['toggle_status'])) {
    $user_id = intval($_GET['toggle_status']);
    if($id != $user_id){   // ← solo compara contra uno mismo, no valida $row['rol']
        $result = mysqli_query($conn, "SELECT active FROM user WHERE user_id = $user_id");
        $row = mysqli_fetch_assoc($result);
        $new_status = $row['active'] ? 0 : 1;
        mysqli_query($conn, "UPDATE user SET active = $new_status WHERE user_id = $user_id");
        // ← se ejecuta igual aunque $user_id sea otro administrador
    }
}
```

RECOMENDACIÓN

✓ REPLICAR EN TOGGLE_STATUS LA MISMA VALIDACIÓN DE ROL QUE YA EXISTE EN EL BLOQUE DE ELIMINACIÓN

```php
if (isset($_GET['toggle_status'])) {
    $user_id = intval($_GET['toggle_status']);
    $stmt = $conn->prepare("SELECT rol, active FROM user WHERE user_id = ?");
    $stmt->bind_param("i", $user_id);
    $stmt->execute();
    $row = $stmt->get_result()->fetch_assoc();

    if ($row['rol'] == 1) {
        $_SESSION['message'] = "No se puede modificar el estado de otro administrador";
        $_SESSION['message_type'] = "error";
    } elseif ($id != $user_id) {
        // ... continuar con la actualización usando prepared statements
    }
}
```

Ausencia de Documentación Técnica y de Usuario del Sistema

IDENTIFICACIÓN: **HAL-017** | RIESGO: **BAJO** | ESTATUS: **NUEVO**

DESCRIPCIÓN

El proyecto no incluye ningún tipo de documentación: no existe `README`, no hay un manual de despliegue, no hay una guía de variables de entorno requeridas (`MAIL_USER`, `RECAPTCHA_SECRET`, credenciales de base de datos), y no hay documentación técnica que describa los módulos, roles (1=administrador, 2=usuario) o el flujo de la bitácora. Esta carencia no es una vulnerabilidad explotable por sí misma, pero es un factor de riesgo indirecto real: es parte de por qué existen HAL-003 y HAL-014 (credenciales y claves hardcodeadas en el código en vez de en variables de entorno documentadas) — sin un estándar documentado de configuración, cada desarrollador resuelve "dónde van las credenciales" a su manera, y la opción más simple termina siendo escribirlas directo en el archivo. También dificulta que un nuevo desarrollador o un futuro auditor entienda el sistema sin leer los 28 archivos PHP uno por uno. OWASP: no mapea a una categoría específica del Top 10 (es una debilidad de proceso, no un vector de ataque directo), pero se relaciona con **A06 — Diseño Inseguro** por la ausencia de estándares documentados de desarrollo seguro, en la misma línea que HAL-013.

```
// Ausente en la raíz del proyecto:
README.md                 // Qué es el sistema, cómo instalarlo y ejecutarlo
.env.example              // Qué variables de entorno se requieren (sin valores reales)
docs/                     // Roles, módulos, diagrama de base de datos

// Consecuencia directa observada en este mismo informe:
// HAL-003 y HAL-014 existen en parte porque no hay un estándar
// documentado de "las credenciales van en variables de entorno"
```

RECOMENDACIÓN

✓ **DOCUMENTAR COMO PARTE DEL MISMO ESFUERZO DE REMEDIACIÓN, NO COMO TAREA SEPARADA**

Agregar un `README.md` con instrucciones de instalación y despliegue; un `.env.example` que liste las variables requeridas sin valores reales (ver corrección de HAL-003); y una guía breve de roles y módulos para que la migración a prepared statements y variables de entorno (HAL-003, HAL-006, HAL-009, HAL-014) quede documentada y no se repita el mismo problema en el futuro.

Ubicación: **Proyecto completo — raíz del repositorio**

SECCIÓN 7

Pruebas de Seguridad

Pruebas Exploratorias · Pruebas de Rendimiento

PRUEBAS EXPLORATORIAS

Se ejecutaron pruebas manuales sobre el sistema en ejecución local (XAMPP), con el objetivo de validar los hallazgos identificados en la revisión de código y detectar vulnerabilidades adicionales en tiempo de ejecución.

PRUEBA	DESCRIPCIÓN	RESULTADO	HAL
SQL Injection — Login	Inyección de payload `' OR '1'='1` en campo email del login	BLOQUEADO — reCAPTCHA previene automatización masiva	—
SQL Injection — Registro	Payload `' OR '1'='1' --` en campo email de add_user.php, ejecutado directamente contra test2.sql (PHP 8.2.12)	NO EXPLOTABLE — ENT_QUOTES (PHP 8.1+) neutraliza la comilla; consulta válida, 0 filas	HAL-006
Acceso directo a SQL	GET http://localhost/buildsafe_system1/test2.sql	VULNERABLE — descarga completa del dump	HAL-010
Debug SMTP visible	Solicitud de recuperación de contraseña con email válido	VULNERABLE — sesión SMTP visible en HTML	HAL-004
Operación GET destructiva	Visitar /list_user.php?delete_id=1 como admin	VULNERABLE — usuario eliminado sin confirmación	HAL-007
CSRF simulado	Formulario externo enviando POST a add_user.php	VULNERABLE — usuario creado exitosamente	HAL-008
Session Fixation	Comparar ID de sesión antes y después del login	VULNERABLE — mismo ID antes y después	HAL-001
Fuerza bruta login	Más de 3 intentos fallidos de contraseña	MITIGADO — sistema bloquea cuenta correctamente	—

PRUEBAS DE RENDIMIENTO

ESCENARIO	MÉTRICA	RESULTADO	EVALUACIÓN
Carga de página login	Tiempo de respuesta	< 200 ms	ÓPTIMO
Listado de usuarios	Tiempo de respuesta (50 registros)	< 300 ms	ÓPTIMO
Registro de movimiento	Tiempo de transacción BD	< 150 ms	ÓPTIMO
Envío de token email	Tiempo total (incl. SMTP)	2-5 seg	ACEPTABLE
Consulta sin índice	JOIN material+movement (sin índice en código)	N/A — no implementado	REVISAR

SECCIÓN 8

Observaciones y Recomendaciones

Visión General · Seguridad · Código · Rendimiento · Legalidad

VISIÓN GENERAL DEL SISTEMA

Aspectos Positivos

BuildSafe System demuestra una base funcional sólida para la gestión de inventario. El sistema implementa correctamente el bloqueo de cuentas tras intentos fallidos de autenticación, lo cual es una medida de seguridad activa. El uso de `password_hash()` y `password_verify()` con bcrypt para la mayoría de los usuarios garantiza un almacenamiento seguro de contraseñas. La integración de Google reCAPTCHA v2 dificulta los ataques automatizados. El módulo de bitácora (`binnacle`) representa una buena práctica de auditoría interna.

Seguridad — Plan de Remediación Priorizado

- **Inmediato (24-48h):** Revocar y regenerar la App Password de Gmail expuesta (HAL-003). Deshabilitar SMTPDebug (HAL-004). Eliminar test2.sql, generar_hash.php del webroot (HAL-010).
- **Corto plazo (1-2 semanas):** Corregir SQL injection en add_user.php con prepared statements (HAL-006). Implementar CSRF en todos los formularios (HAL-008). Cambiar operaciones GET a POST en list_user.php, list_material.php y list_materials2.php — ambas interfaces de rol 1 y rol 2 (HAL-007). Migrar list_material.php a prepared statements igual que su par list_materials.php/list_materials2.php (HAL-009). Replicar en toggle_status la validación de rol que ya existe en el bloque de eliminación (HAL-016).
- **Mediano plazo (1 mes):** Agregar session_regenerate_id() tras login (HAL-001). Migrar todas las consultas a prepared statements (HAL-009). Implementar rate limiting en recuperación (HAL-005).
- **Mantenimiento (próximo ciclo):** Agregar cabeceras HTTP de seguridad (HAL-015). Mover credenciales a variables de entorno (HAL-003, HAL-014). Corregir error lógico en head.php (HAL-002). Documentar instalación, variables de entorno y roles del sistema (HAL-017).

Código Fuente y Arquitectura

Se recomienda adoptar un patrón de arquitectura MVC (Model-View-Controller) o al menos separar la lógica de negocio de la presentación. Los archivos actuales mezclan HTML, PHP y SQL en las mismas vistas, lo que dificulta la revisión de seguridad y el mantenimiento. Se sugiere crear un directorio `includes/` para la capa de acceso a datos, utilizando exclusivamente prepared statements.

Control de Versiones y Documentación

Inicializar un repositorio Git con un `.gitignore` que excluya `.env`, backups SQL y archivos de configuración locales. Documentar el proceso de despliegue y las variables de entorno requeridas. Establecer un flujo de revisión de código (code review) antes de cualquier merge a producción.

Consideraciones Legales — Venezuela

Las vulnerabilidades críticas identificadas, en particular la exposición de contraseñas y datos de usuarios, pueden comprometer el cumplimiento de las disposiciones del **Decreto con Fuerza de Ley sobre Mensajes de Datos y Firmas Electrónicas (2001)** y la **Ley Especial contra los Delitos Informáticos (2001)** de Venezuela, que establece responsabilidades para los custodios de datos electrónicos. Se recomienda corrección inmediata previo a cualquier despliegue en ambiente de producción con datos reales.

TABLA CONSOLIDADA DE HALLAZGOS

ID	VULNERABILIDAD	SEVERIDAD	ARCHIVO	OWASP	ESFUERZO
HAL-001	Sin regeneración de sesión	MEDIO	index.php:88	A07	Muy Bajo
HAL-002	Error lógico verificación de rol	MEDIO	head.php:12	A07	Muy Bajo
HAL-003	Credenciales SMTP hardcodeadas	CRÍTICO	enviar_token.php:37	A07	Bajo
HAL-004	Debug SMTP nivel 3 en producción	CRÍTICO	enviar_token.php:41	A09	Muy Bajo
HAL-005	Sin rate limit recuperación	MEDIO	enviar_token.php	A06	Bajo
HAL-006	Construcción insegura de consulta SQL (no explotable en PHP 8.2)	MEDIO	add_user.php:29	A05	Bajo
HAL-007	Operaciones destructivas vía GET	ALTO	list_user.php:5,46 · list_material.php:5,30 · list_materials2.php:5	A01	Bajo
HAL-008	Sin protección CSRF	ALTO	Todos los formularios	A01	Medio
HAL-009	SQL sin prepared statements	ALTO	list_user.php:52,65 · list_material.php:34,40,47	A05	Bajo
HAL-010	Archivos sensibles en webroot	ALTO	test2.sql, *.php	A02	Muy Bajo
HAL-011	Errores BD expuestos	MEDIO	add_user.php:56	A10	Muy Bajo
HAL-012	Contraseña sin hash en BD	MEDIO	test2.sql (user_id=9)	A04	Muy Bajo
HAL-013	Sin control de versiones	MEDIO	Proyecto completo	A06	Bajo
HAL-014	reCAPTCHA secret hardcodeada	BAJO	index.php:47	A02	Muy Bajo
HAL-015	Sin cabeceras de seguridad HTTP	BAJO	head.php	A02	Muy Bajo
HAL-016	Falta de verificación de rol en cambio de estado	ALTO	list_user.php:46-58	A01	Muy Bajo
HAL-017	Ausencia de documentación técnica	BAJO	Proyecto completo	A06	Medio

CONCLUSIONES FINALES

La auditoría de seguridad realizada al sistema BuildSafe System de CONFALVI C.A. evidencia que el sistema posee una **postura de seguridad deficiente** que impide su despliegue responsable en un entorno de producción accesible públicamente en su estado actual. Los 17 hallazgos identificados abarcan las capas de autenticación, manejo de datos, configuración del servidor, gestión de credenciales y control de acceso.

⚠ VEREDICTO: NO APTO PARA PRODUCCIÓN

El sistema presenta 2 vulnerabilidades críticas activas que deben ser corregidas antes de cualquier despliegue. La exposición de credenciales SMTP y el dump de base de datos accesible vía HTTP representan riesgos inmediatos y explotables sin necesidad de conocimientos técnicos avanzados. Adicionalmente, se identificó un patrón de construcción de consultas SQL inseguro (HAL-006) que, si bien no resultó explotable en la versión de PHP actualmente instalada, debe corregirse por no constituir una defensa real.

LO QUE FUNCIONA BIEN

- Bloqueo de cuentas tras intentos fallidos
- Hash bcrypt para la mayoría de contraseñas
- reCAPTCHA v2 en el login
- Bitácora de auditoría de acciones
- Separación de roles admin/usuario
- Transacciones en movimientos de inventario

REQUIERE CORRECCIÓN INMEDIATA

- Revocar App Password de Gmail expuesta
- Deshabilitar SMTPDebug
- Eliminar test2.sql del webroot
- Migrar la consulta de duplicados de add_user.php a prepared statements (HAL-006)
- Implementar protección CSRF
- Cambiar acciones destructivas de GET a POST
- Validar rol de administrador en toggle_status (HAL-016)

Una vez corregidos los hallazgos críticos y altos, se recomienda realizar una segunda auditoría de validación para confirmar la efectividad de los controles implementados. Con las correcciones aplicadas, BuildSafe System tiene el potencial de convertirse en un sistema robusto y seguro para la gestión de inventario de CONFALVI C.A.

DOCUMENTO GENERADO POR

Auditoría de Seguridad Informática

Universidad Politécnica Territorial Alonso Gamero · CONFALVI C.A. · Julio 2026

CONFIDENCIAL — USO INTERNO EXCLUSIVO